

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

ML-based Deep Bitstream Inspection for serverless FPGAs

Semester Project

Santiago de Heredia Tenorio
sdeheredia@ethz.ch

Systems Group
ETH Zürich

Supervisors:
Maximilian Heer
Prof. Dr. Gustavo Alonso

July 17, 2026

Acknowledgements

I would like to thank Maximilian Heer and Prof. Dr. Gustavo Alonso for their guidance and supervision throughout this semester project. I am also grateful to the Systems Group at ETH Zürich for the project context, infrastructure, and feedback.

Abstract

Cloud and serverless FPGA platforms expose reconfigurable fabric to user designs, making them vulnerable to malicious circuits such as ring oscillators [1, 2, 3]. Prior work on ML-based bitstream inspection has shown that malicious payloads can be detected from bitstream-level patterns [4, 5, 6, 7], but these approaches are generally evaluated as offline software classifiers and are not designed for low-latency deployment on datacenter-grade FPGA infrastructure. This motivates an online inspection approach in which the detection model itself is implemented in FPGA fabric and integrated into a cloud-oriented FPGA shell before user bitstreams are deployed.

This report investigates whether malicious-bitstream detection can be moved into the FPGA deployment path itself. We generate a dataset of benign Coyote partial bitstreams and malicious ring-oscillator bitstreams, represent bitstreams as image-like inputs, train compact CNN classifiers, and synthesize selected models using hls4ml [8, 9]. We evaluate the resulting design space across classification quality, input resolution, model depth, quantization, pruning, reuse factor, latency, and FPGA resource utilization.

Our results show that the Alveo U55C 256×7 W8A8 P50 manual-v1 checker reaches 87.7% accuracy, 88.5% F1, and 8.0% false-negative rate on the held-out test set. The alternative 512×7 W8A8 P50 manual-v1 checker reaches 90.4% accuracy, 90.5% F1, and 10.7% false-negative rate. Estimated FPGA-side preprocessing-plus-inference latency is 3.35 ms per sample for 256×7 and 6.53 ms per sample for 512×7. The 256×7 routed design attributes 4.47% LUT, 17.86% BRAM, 1.15% URAM, and 14.72% DSP to the checker wrapper. The 512×7 design attributes 4.90% LUT, 61.78% BRAM, 1.77% URAM, and 14.45% DSP. These results suggest that FPGA-resident ML-based inspection is feasible as a deployment-path defense-in-depth checker for RO-payload detection, but remains limited by dataset breadth, low-footprint attacks, threshold policy, and full production control-plane integration.

Contents

Acknowledgements i

Abstract ii

1 Introduction 1

2 Background, Threat Model, and Gap 3

2.1 Serverless FPGAs and Partial Reconfiguration 3

2.2 Ring-Oscillator Payloads 3

2.3 Threat Model 4

2.4 Prior Work 4

2.5 Research Gap 5

3 Dataset and Preprocessing 7

3.1 Dataset Goal 7

3.2 Bitstream Generation 7

3.3 Dataset Composition 8

3.4 Preprocessing 9

3.5 Dataset Limitations 10

4 CNN Inspection Pipeline 12

4.1 Choice of CNNs 12

4.2 Model Input and Classification Task 12

4.3 Design Space 13

4.4 Candidate Architecture and Training 13

4.5 Software Detection Results 14

4.6 Interpretability and Failure Modes 16

4.7 Extended RO-Footprint Evaluation 17

5 hls4ml Synthesis and Hardware Results **19**
5.1 Synthesis Goal . . . 19
5.2 hls4ml Flow . . . 19
5.3 Hardware Tuning Parameters . . . 20
5.4 Latency and Resource Tradeoff . . . 20
5.5 Final Synthesized Candidates . . . 21
5.6 Latency Context . . . 22
5.7 Resource Context . . . 23
5.8 Quantization and Pruning . . . 23
5.9 Takeaway . . . 24

6 Proposed Coyote Integration **26**
6.1 Integration Point . . . 26
6.2 Checker Flow . . . 26
6.3 Prototype Scope . . . 27

7 Discussion, Limitations, and Future Work **28**
7.1 Discussion . . . 28
7.2 Limitations . . . 28
7.3 Future Work . . . 29

8 Conclusion **31**

Bibliography **32**

Chapter 1

Introduction

Field-programmable gate arrays (FPGAs) are increasingly used as datacenter accelerators because they provide programmable hardware with high performance and energy efficiency [10, 11, 12, 13]. In cloud and serverless settings, this programmability enables users to submit hardware applications that are deployed on shared FPGA infrastructure [14]. Partial reconfiguration (PR) makes this model practical by allowing user logic to be loaded into a reconfigurable region while the trusted FPGA shell remains active [15, 16].

This deployment model creates a security boundary. The user partial bitstream becomes an input to the platform, and loading it into the FPGA fabric may expose shared physical resources such as power delivery, thermal headroom, routing infrastructure, and neighboring workloads. Malicious circuits based on ring oscillators (ROs) are a representative threat: at scale, RO arrays can generate high-frequency switching activity, waste power, induce voltage drops, and cause denial-of-service or fault effects [1, 17, 2, 3, 18, 19].

Existing cloud FPGA platforms show that bitstream validation is a practical concern. Prior attacks on AWS F1 demonstrated that malicious power-hammering designs could pass platform checks and crash instances [2]. AWS F2 continues the cloud-FPGA deployment model, and its official flow supports development and deployment of reconfigurable hardware on second-generation FPGA instances [20]. This keeps deployment-path validation relevant for newer cloud FPGA platforms.

Machine-learning-based bitstream inspection is a promising direction because prior work has shown that bitstream-derived representations contain learnable structure and can be used to classify malicious designs [4, 5, 21, 6, 7]. However, most ML-based approaches are evaluated as offline software classifiers. This leaves a systems gap: a serverless FPGA platform needs a checker that fits the deployment path. Such a checker should be accurate enough to reject suspicious bitstreams, small enough to coexist with platform and user logic, and fast enough to avoid dominating PR latency.

This project studies whether ML-based malicious-bitstream inspection can

move into the FPGA deployment path itself. Instead of relying on host-side CPU or GPU inspection, we explore a compact FPGA-resident checker for Coyote-style serverless FPGA systems [15, 16]. The checker receives a user partial bitstream, applies a deterministic byte-stream preprocessing step, runs a synthesized CNN classifier, and produces an accept/reject decision or suspiciousness score before PR.

We evaluate this idea using a dataset of benign Coyote application partial bitstreams and RO-based malicious partial bitstreams. Each bitstream is transformed into a fixed-size image-like tensor through deterministic resizing. We then train CNN classifiers, explore input resolution and model depth, synthesize selected candidates with hls4ml [8, 9], and evaluate the resulting hardware designs across detection quality, false-positive and false-negative behavior, latency, and FPGA resource overhead.

This report makes four contributions:

1. **Partial-bitstream dataset.** We construct a dataset of Coyote partial bitstreams from real benign Coyote applications and RO-based malicious payloads.
2. **Datacenter FPGA setting.** We evaluate inspection in a datacenter deployment setting with partial reconfiguration and UltraScale+ FPGA hardware.
3. **ML-to-hardware inspection path.** We connect CNN training to hls4ml synthesis and evaluate whether trained bitstream classifiers can become practical FPGA-resident checkers.
4. **Hardware-aware design-space evaluation.** We compare candidate models across detection quality, FNR/FPR, ROC AUC, LUT, BRAM, DSP, and added inspection latency.

The main result is that FPGA-resident ML bitstream inspection is feasible as a deployment-path defense-in-depth mechanism. Among the synthesized candidates, the 256×7 CNN provides the preferred deployment point because it balances useful detection quality with lower latency and substantially lower BRAM overhead than larger alternatives. Overall, the prototype shows a practical path from user-submitted partial bitstreams to a hardware-backed checker before deployment.

Chapter 2

Background, Threat Model, and Gap

2.1 Serverless FPGAs and Partial Reconfiguration

Serverless FPGA systems expose reconfigurable hardware as an on-demand acceleration service. Users submit hardware applications, while the platform manages the FPGA shell, host interface, memory system, networking, and runtime services [15, 16, 14]. This model hides low-level FPGA management from users and allows the provider to share FPGA capacity across applications.

Partial reconfiguration makes this deployment model practical. The FPGA is divided into a static region and one or more reconfigurable regions. The static region contains trusted platform logic, while the reconfigurable region contains user logic. A user application can therefore be loaded without rebuilding the full FPGA image.

This structure creates a natural security boundary. The user partial bitstream enters the platform before it is loaded into the reconfigurable region. If the bitstream contains malicious logic, the platform may configure that logic directly into shared FPGA fabric. The deployment path should therefore include an inspection step before partial reconfiguration.

Coyote is a relevant target for this work because it provides a datacenter FPGA shell with partial reconfiguration, multitenancy, and platform services [15, 16]. It gives a concrete setting for studying whether bitstream inspection can become part of the FPGA deployment path.

2.2 Ring-Oscillator Payloads

Ring oscillators are a simple and well-studied malicious payload for FPGAs [1, 17, 2, 3]. A ring oscillator is built from an odd feedback loop of inverters. The loop has no stable state and continuously toggles. A single ring oscillator is small,

but a large array of attacker-controlled ring oscillators can create high-frequency switching activity.

This switching activity stresses shared physical resources. In a cloud or serverless FPGA setting, the relevant resources include power delivery, thermal headroom, and neighboring user logic. Large ring-oscillator arrays can waste power, induce voltage drops, cause denial of service, or trigger timing faults. For this reason, RO-based payloads are a useful first target for malicious-bitstream inspection.

This work focuses on detecting RO-based malicious partial bitstreams. This is a narrow threat class, but it is realistic enough to test the core systems question. The main goal is to evaluate whether a learned detector can be accurate, small, and fast enough to sit in the deployment path.

2.3 Threat Model

We consider a serverless FPGA platform with a trusted shell and an untrusted user partial bitstream. The attacker can submit a syntactically valid partial bitstream for the user reconfigurable region. The bitstream may contain an RO-based payload designed to stress shared physical resources after configuration.

The attacker's goal is to have the malicious bitstream accepted and loaded through partial reconfiguration. The platform's goal is to inspect the bitstream before configuration and reject suspicious inputs.

The checker is placed before the partial reconfiguration step. It receives the incoming partial bitstream, preprocesses it into the fixed input format expected by the classifier, and outputs a decision.

2.4 Prior Work

FPGA security surveys provide the broader context for bitstream-level attacks and defenses [22, 23]. For this report, the most relevant defenses fall into three groups. Cloud and vendor checks are practical because they are integrated into deployment flows, but they usually target known rule violations [24, 2]. Structural methods search for suspicious circuit patterns, such as self-oscillators, but can depend strongly on device architecture and bitstream structure [17, 3, 25]. ML-based methods learn patterns from bitstream-derived data, but are commonly evaluated as offline software classifiers [4, 5, 6, 7, 26].

Prior ML-based work is important because it shows that bitstream-derived representations contain learnable structure [4, 5, 6, 7]. CNNs and other classifiers can distinguish benign and malicious bitstreams without fully decoding the circuit semantics. Recent work by Stahle-Smith and Karakchi is especially close in

Table 2.1: Comparison of approaches for malicious FPGA bitstream detection.

Approach	**Strength**	**Limitation**	**Relation to this work**
Cloud and vendor checks	Already integrated into deployment flows	Restricted to known rules and platform policies	Complemented by learned inspection
Structural RO analysis	Detects oscillator structures directly	Often architecture-specific and difficult to generalize	Avoids explicit structural reverse engineering
ML-based inspection	Learns patterns directly from bitstream data	Commonly evaluated as an offline software classifier	Moves the learned detector into FPGA fabric
This work	Connects partial-bitstream classification with hardware synthesis	Evaluates one platform and one primary threat class	Studies detection quality, hardware cost, and deployment-path integration

motivation because it uses static byte-level features for malicious-bitstream detection and deploys the selected classifier on a PYNQ-Z1 embedded platform [26]. That work uses byte-frequency features, TSVD, SMOTE, and a Random Forest for embedded-system detection. This report instead studies deterministic byte-image tensors, CNN classifiers, hls4ml synthesis, and integration into a Coyote-style U55C deployment path. The remaining question is whether such inspection can satisfy the constraints of a serverless FPGA deployment path.

2.5 Research Gap

The gap addressed in this report is deployment-oriented ML inspection. Offline classification alone is not enough for a serverless FPGA platform. A useful checker must satisfy three requirements.

Table 2.2: Feasibility dimensions for FPGA-resident bitstream inspection.

Dimension	**Question**	**Metrics**
Detection quality	Does the checker retain useful security coverage?	F1, FNR, FPR, ROC AUC
Hardware cost	Can the checker fit beside platform and user logic?	Latency, LUT, BRAM, DSP
Integration	Can the checker sit on the Coyote partial-reconfiguration path?	PR-path placement and prototype design

This report studies all three dimensions together. We generate Coyote-style partial bitstreams, train CNN classifiers on deterministic byte-image representations, synthesize selected models with hls4ml, and evaluate whether the resulting hardware designs are practical candidates for deployment-path inspection.

Chapter 3

Dataset and Preprocessing

3.1 Dataset Goal

The goal of the dataset is to evaluate malicious-bitstream inspection on the same type of artifact that appears in a serverless FPGA deployment path. The input to the checker is a partial bitstream, not source code, a netlist, or an implementation report. The dataset is therefore built from Coyote-style partial bitstreams [15, 16].

The classification task is binary. Benign samples are partial bitstreams generated from Coyote applications. Malicious samples are partial bitstreams that contain ring-oscillator payloads. Labels are assigned by construction from the generation flow.

This dataset is designed to test deployment-like variation. Samples are generated across different floorplans and configurations so that the classifier is not evaluated on a single fixed placement. The malicious class also includes higher-density RO samples to evaluate how detection changes as the RO footprint increases.

3.2 Bitstream Generation

Benign samples are generated from real Coyote applications. These applications represent normal user accelerators that could be deployed into a reconfigurable region. To increase variation, the applications are built across multiple floorplans and configurations.

The benign catalog contains eight debug-enabled Coyote applications: hello world, HLS vector add, multitenancy AES, user interrupts, perf FPGA, multithreading AES, Euclidean distance, and cosine similarity. It also contains no-debug variants for seven of these applications. In the available manifests, this gives 120 debug-enabled benign samples and 101 no-debug benign samples.

Malicious samples are generated from RO designs from [19]. The RO count

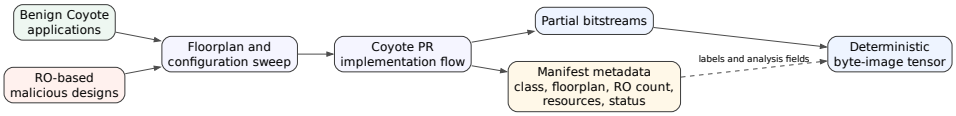


Figure 3.1: Dataset generation pipeline. Benign Coyote applications and RO-based malicious designs are generated across deployment-like variation and converted into labeled partial bitstreams.

is varied to create payloads with different footprints [17, 3, 21]. This allows the evaluation to distinguish between small low-footprint payloads and larger high-density payloads.

Figure 3.1 shows the dataset generation flow. Both benign and malicious designs pass through a floorplan and configuration sweep before partial-bitstream generation. The output is a labeled dataset of partial bitstreams and metadata.

3.3 Dataset Composition

Table 3.1 summarizes the available generated dataset. The original generated corpus contains benign Coyote bitstreams and normal-RO malicious bitstreams across 15 floorplans, FP00–FP14. The higher-density big-RO extension uses six of those floorplans and is kept separate because it is used for the extended footprint evaluation. Figure 3.2 shows the RO-count distribution for both malicious subsets.

The available generated manifests contain 221 benign samples, 225 normal-RO malicious samples, and 78 big-RO malicious samples. The normal-RO corpus spans 32 RO-count settings. The big-RO extension spans 13 higher-density RO-count settings. These counts are not full Cartesian products. For normal RO, we did not build every RO setting on every floorplan. Instead, the generation flow used more RO-count settings and a subset of floorplan combinations to keep the malicious sample count close to the benign count. For benign samples, the intended catalog is 15 variants across 15 floorplans, but some builds did not finish, leaving 221 available benign bitstreams.

For training and held-out testing, benign samples are always kept, while standalone RO samples are kept only when `ro_count` is at least 8000. We use this threshold as a project-level lower bound for attack-relevant RO payloads. Prior work such as FPGADefender shows that malicious self-oscillator payloads at this range (particularly as a percentage of LUTs occupied by ROs) are a realistic FPGA security target [17]; the 8000-RO cutoff is our dataset policy to establisj a lower bound for attack-relevant RO payloads.

Each sample also stores metadata used during analysis. This includes the

Table 3.1: Dataset composition by class and RO category.

Class	Source	Variation	Floorplans	Samples
Benign	Coyote applications	8 apps, 7 no-debug variants	15	221
Normal RO	Standalone RO payloads	32 RO-count settings	15	225
Big RO	Higher-density RO payloads	13 RO-count settings	6	78

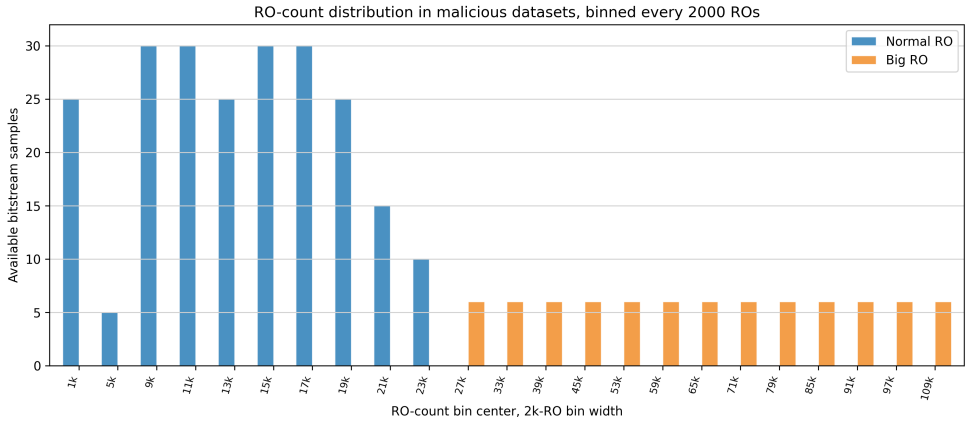


Figure 3.2: Distribution of RO counts in the normal-RO dataset and the big-RO extension. Bars count available bitstream samples in 2k-RO bins.

design source, floorplan, configuration, label, bitstream size, and estimated RO footprint when applicable. The RO footprint metadata is used later to evaluate whether misses concentrate in small malicious designs.

3.4 Preprocessing

The preprocessing step converts each partial bitstream into the fixed input shape expected by the CNN. Related work has used bitstream-derived tensors or image-like transforms for FPGA classification tasks [4, 5, 27]. Our transform treats the entire `.bin` file as a byte stream. It does not skip headers, reverse-engineer the bitstream, or extract hand-crafted circuit features.

Algorithm 1 is the exact preprocessing rule. Long bitstreams are sampled uniformly across the full byte range using deterministic `linspace` indices, rather than truncated. Short bitstreams are padded with raw zero bytes before inversion, so padding becomes normalized value 1.0 after division by 255. The rule is deterministic at every resolution, although each resolution has its own target length and sampling grid.

Algorithm 1 Pseudocode for deterministic byte-image preprocessing.

Require: bitstream path p, image height H, image width W
$T \leftarrow H \cdot W$
$b \leftarrow \text{ReadAllBytesAsUint8}(p)$
if $|b| > T$ **then**
    $i \leftarrow \lfloor \text{Linspace}(0, |b| - 1, T) \rfloor$
    $w \leftarrow b[i]$
else
    $w \leftarrow \text{ZerosUint8}(T)$
    $w[0 : |b|] \leftarrow b$
end if
$w \leftarrow 255 - w$
$x \leftarrow \text{Reshape}(w, H, W, 1)$
$x \leftarrow x/255.0$
return x

The main input resolutions are

$$128^2, \quad 256^2, \quad 512^2.$$

Larger resolutions are also used during model exploration to measure how much classification quality improves when more bitstream detail is preserved.

This representation is useful for two reasons. First, it gives CNNs a fixed-size input while preserving byte-level structure from across the bitstream. Second, the operation maps directly to FPGA logic before inference. We wrote this preprocessing block as HLS C++ raw-byte downsampling logic, lowered it to Verilog through the HLS flow, and used it in the deployment-oriented latency results reported in Chapter 5. This matters because the final goal is an FPGA-resident checker that can run in the deployment path.

The final latency numbers in Chapter 5 include the estimated FPGA raw-byte downsampler latency and the hls4ml CNN latency. They exclude Python, host setup, memory-copy overhead, and output readback. The routed resource percentages report the checker wrapper/user-logic attribution. They do not isolate the downsampler resources from the CNN resources.

3.5 Dataset Limitations

The dataset supports a feasibility study. It does not cover every malicious FPGA design. The malicious class focuses on RO-based payloads, and labels are assigned by construction. This means the classifier may learn artifacts that correlate with maliciousness, such as payload size, routing density, or floorplan effects.

The extended RO-footprint evaluation helps address this issue by showing where the detector is strong and where it is weak. Small RO payloads remain the harder case. Larger RO payloads are more relevant for power-stress attacks and are evaluated separately in Section 4.

Chapter 4

CNN Inspection Pipeline

4.1 Choice of CNNs

We use convolutional neural networks because prior work has shown that representations derived from bitstreams contain learnable structure. Rettkowski et al. studied neural-network-based inspection of partial FPGA bitstreams. Chaudhuri and Chakrabarty used CNN-based learning to detect malicious FPGA bitstreams. Elnaggar et al. also used CNNs as part of a malicious-circuit detection pipeline. These results support the idea that bitstreams can be treated as structured data for classification, even without full bitstream reverse engineering [4, 5, 21, 28, 6, 7].

This project follows the same direction, but with a different systems goal. The model is not only trained for software accuracy. It must also be small enough and regular enough to synthesize into FPGA logic. CNNs are a good fit for this because they can operate on fixed-size image-like tensors and can be converted to hardware using tools such as hls4ml [8, 9].

4.2 Model Input and Classification Task

Each partial bitstream is converted into a grayscale image-like tensor through deterministic resizing. This gives the CNN local neighborhoods of byte values from which it can learn statistical and spatial patterns.

The main classification task is binary:

$$y = \begin{cases} 0, & \text{benign Coyote partial bitstream,} \\ 1, & \text{RO-based malicious partial bitstream.} \end{cases}$$

The main metrics are accuracy, F1, true positive rate, false positive rate, false negative rate, and ROC AUC. False negatives are especially important because they correspond to malicious bitstreams that would be accepted by the checker.

Table 4.1: CNN design-space dimensions.

Dimension	**Question**	**Main metrics**
Input resolution	How much bitstream detail is retained	F1, FNR, ROC AUC
CNN depth	How much model capacity is needed	F1, TPR, FPR
Quantization	How little precision is enough	AUC, hls4ml parity, resources
Pruning	How much sparsity can be introduced	AUC, sparsity, synthesis impact

4.3 Design Space

The CNN design space is explored in stages. The goal is to preserve useful detection quality while keeping the final hardware design practical. We therefore vary the model only along dimensions that affect both classification and hardware cost.

The main software sweep varies input resolution and depth. The tested resolutions include smaller inputs such as 128^2 and larger inputs such as 256^2 and 512^2. Higher resolutions preserve more bitstream detail, but they also increase the cost of the synthesized checker. The depth sweep evaluates whether additional convolutional layers improve detection enough to justify the hardware cost.

4.4 Candidate Architecture and Training

The synthesized 256×7 and 512×7 candidates use the same seven-stage convolutional architecture. They differ in input resolution and final average-pool area. The 256×7 model pools over a 1×1 final feature map. The 512×7 model pools over a 2×2 final feature map. The output layer has one logit. Probabilities are computed by applying a sigmoid to this logit. The reported default predictions use threshold 0.5 on probability, equivalent to threshold 0 on the logit.

The final candidates are the 256×7 and 512×7 W8A8 P50 manual-v1 models. W8A8 means 8-bit quantized weights and activations. P50 means pruning with a final sparsity target of 0.5 on pruned layers, while the output dense layer is not pruned. Manual-v1 refers to the manually tuned per-layer hls4ml configuration used for the deployment evaluation.

Each convolution uses explicit zero padding before a valid convolution. No

Table 4.2: Seven-stage QKeras CNN architecture used for the final candidates.

Stage	Layer	Filters	Kernel	Stride	Pooling	Activation	Quantization
Input	Byte image	–	–	–	–	–	float input
0	QConv2D	8	5×5	2×2	MaxPool 2×2	QReLU	W8 / A8(2)
1	QConv2D	16	3×3	1×1	MaxPool 2×2	QReLU	W8 / A8(2)
2	QConv2D	24	3×3	1×1	MaxPool 2×2	QReLU	W8 / A8(3)
3	QConv2D	24	3×3	1×1	MaxPool 2×2	QReLU	W8 / A8(4)
4	QConv2D	32	3×3	1×1	MaxPool 2×2	QReLU	W8 / A8(5)
5	QConv2D	32	3×3	1×1	MaxPool 2×2	QReLU	W8 / A8(5)
6	QConv2D	32	3×3	1×1	MaxPool 2×2 + final avg.	QReLU	W8 / A8(5)
Output	QDense	1	–	–	–	linear logit	W8

batch normalization is used. Each convolutional stage is followed by quantized ReLU activation and 2×2 max pooling. A final average-pool layer and flattening step feed the dense output. Hardware resource use is reported in Chapter 5.

The production training setup is:

- **Framework and model path:** QKeras with TensorFlow, `tf_keras`, and TensorFlow Model Optimization.
- **Optimizer and loss:** Adam with learning rate 10^{-4} and binary cross-entropy from logits.
- **Batch and epochs:** batch size 16 and a fixed 300-epoch budget.
- **Seeds:** seed 42 for dataset splitting and seed 10042 for production-model initialization.
- **Split protocol:** training uses a class-balanced subset created by down-sampling benign samples to match the available RO-malicious samples. Model selection uses stratified five-fold validation with seed 42. The final production model is retrained on the full balanced training subset and evaluated once on a separate held-out test set.
- **Class weights:** no class weights are used after balancing.
- **Augmentation:** horizontal and vertical flips with probability 0.5 each, no translation, and no random crop scaling below 1.0.
- **Thresholds:** We use a probability threshold of 0.5 for decisions (higher means a more suspicious bitstream).
- **Reproducibility metadata:** production manifests record source hashes.

4.5 Software Detection Results

Figure 4.1 shows the main (software) machine learning result:

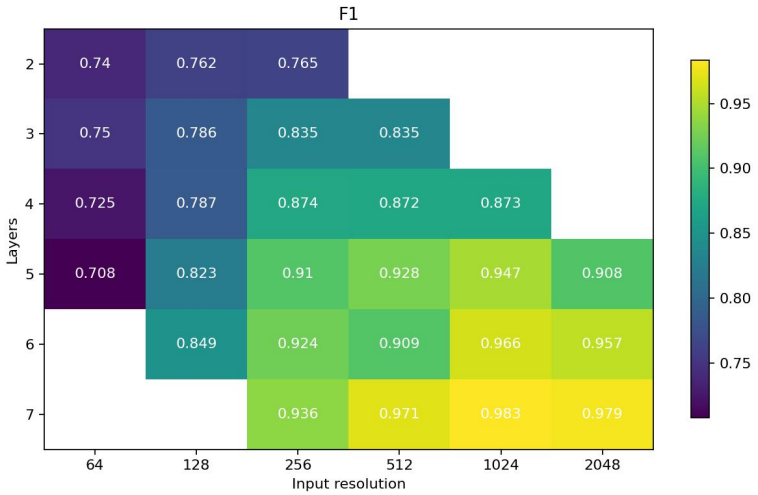


Figure 4.1: F1 score across input resolution and CNN depth. Higher resolution and deeper models improve software detection quality, but they also increase hardware cost.

Table 4.3: Representative pooled-validation software CNN results.

Model	Resolution	Depth	F1	Role
256×7	256^2	7	93.6%	Deployment candidate
512×7	512^2	7	97.1%	Accuracy-oriented candidate
1024×7	1024^2	7	98.3%	Software upper-bound candidate

- F1 improves as input resolution and model depth increase.
- Small inputs lose information during resizing, while larger inputs preserve more byte-level structure.
- Deeper models improve classification up to the evaluated range.

The strongest software models are not necessarily the best candidates. Larger inputs improve F1, but they also increase latency and resource usage after synthesis. The later hardware evaluation therefore focuses on deployable candidates rather than only the best F1 software model.

Note: Table 4.3 and Table 5.2 should not be read as the same metric on the same data. Table 4.3 reports pooled validation performance from the software design-space sweep. Table 5.2 reports final production candidates on a held-out test set after pruning, quantization, hls4ml conversion, and U55C validation. This split and flow change explains why the final 256×7 and 512×7 F1 values are lower than the software sweep values.

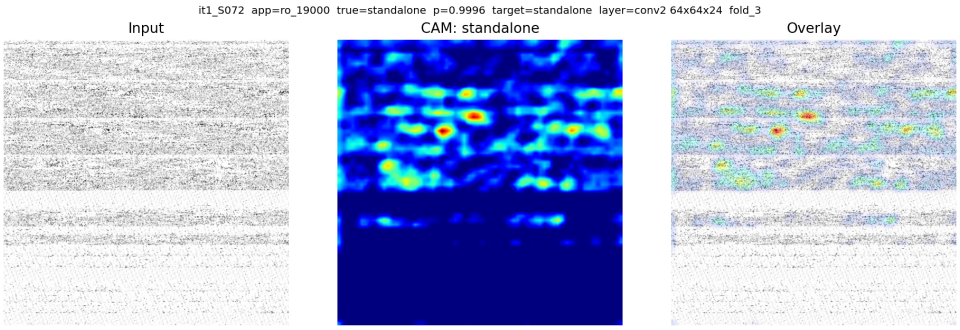


Figure 4.2: Grad-CAM example for a correctly classified RO-malicious sample. The input, class activation map, and overlay show localized regions that contribute to the standalone prediction.

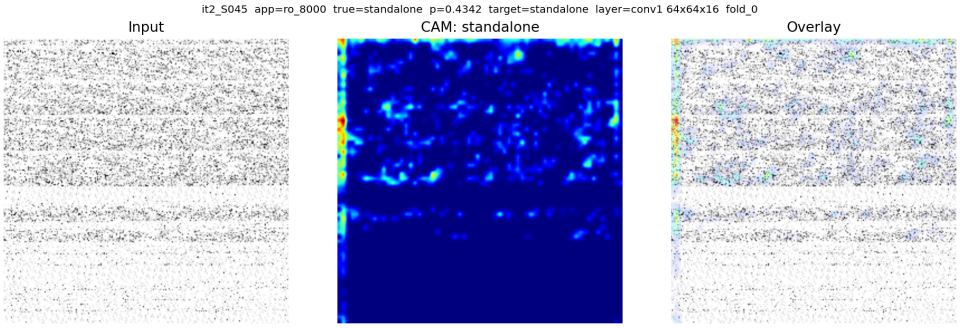


Figure 4.3: Grad-CAM failure example for the 256×7 fold 0 model. The RO-malicious `it2_S045` sample is predicted benign at the default threshold, and the heatmap concentrates on linear bitstream patterns.

4.6 Interpretability and Failure Modes

We use Grad-CAM as a qualitative sanity check [29]. The goal is to inspect whether the CNN focuses on localized regions of the bitstream image or behaves as if it only learned a trivial global statistic.

The Grad-CAM examples show localized regions and bands with higher contribution. This supports the use of a spatial bitstream representation. However, it does not prove that the model identifies ring oscillators at the circuit level, and it is only a qualitative check on the learned representation.

Grad-CAM can also expose a failure mode. In fold 0 of the 256×7 k-fold CV, the sample `it2_S045` is a RO design with 8000 ROs, but the model assigns standalone probability 0.434 and predicts benign. Figure 4.3 shows that the activation concentrates on linear edge and band patterns rather than clear distributed evidence of the malicious payload.

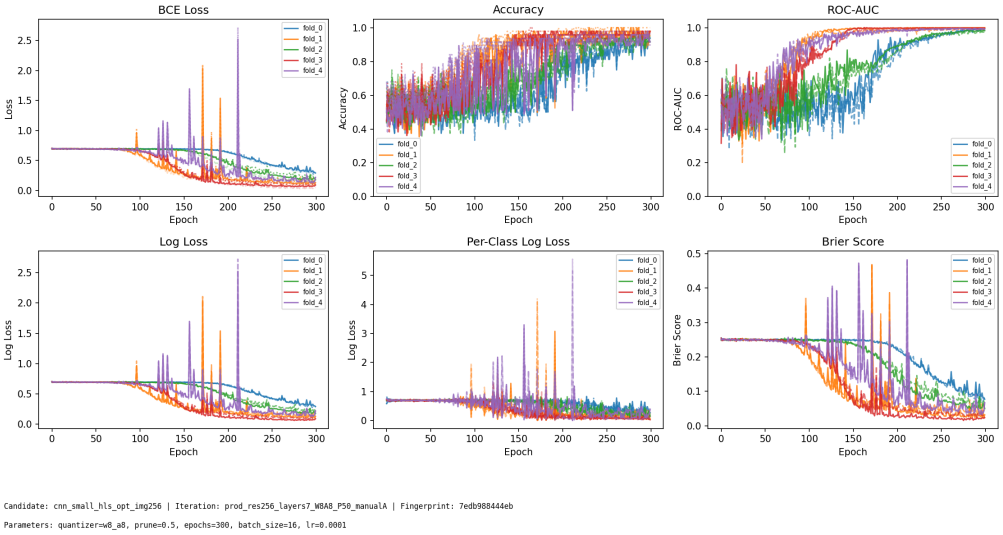


Figure 4.4: Training diagnostics for the 256×7 W8A8 P50 run. Fold 0 shows slower improvement and a higher loss trajectory than the strongest folds, motivating additional checks beyond qualitative Grad-CAM inspection.

The training curves provide the corresponding warning. In Figure 4.4, fold 0 learns more slowly than the other folds and its loss remains comparatively high. We interpret this as an overfitting or training-instability issue that needs detection beyond Grad-CAM. This matters most for lower-signal settings such as the non-extended dataset, where a model can fit onto incidental linear bitstream patterns instead of robust malicious-payload evidence. Practical use should therefore pair Grad-CAM with fold-level diagnostics, per-sample failure review, threshold monitoring, and retraining or stronger regularization when a fold shows this behavior.

4.7 Extended RO-Footprint Evaluation

The original dataset tests whether the classifier can separate benign Coyote bitstreams from RO-based malicious bitstreams. We also evaluate how detection changes as the RO footprint increases. This matters because high-density RO payloads are more relevant for power-stress and timing-fault scenarios.

Figure 4.5 shows the extended RO-footprint result as binned detection rates. Misses concentrate in the smallest RO-footprint bin. Detection becomes reliable for larger evaluated payloads. This suggests that the classifier is strongest for high-density RO attacks, while low-footprint payloads remain the harder case.

This result is important for interpreting the detector. The model does not provide complete coverage against all malicious circuits. It provides useful cov-

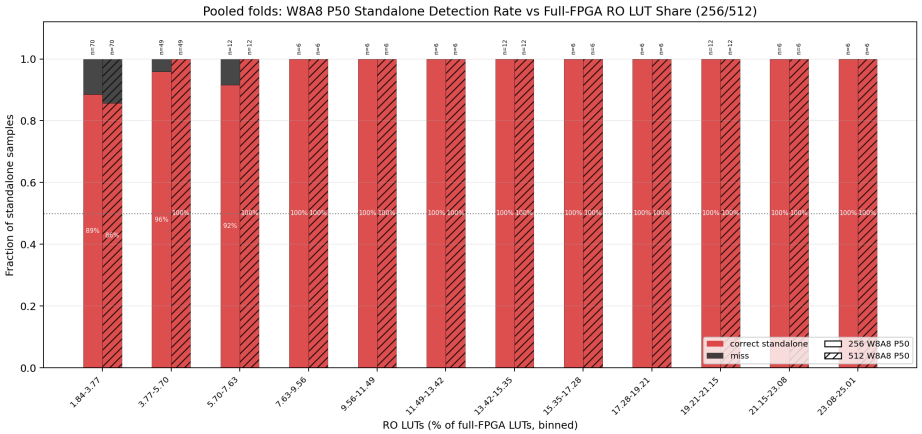


Figure 4.5: Binned standalone detection rate versus estimated RO LUT share for W8A8 P50 256×7 and 512×7 models. The smallest RO-footprint bins contain most misses, while larger evaluated RO payloads are detected reliably.

erage for RO-based payloads, especially larger payloads that are more likely to create practical power-stress effects. The next chapter evaluates whether these CNN candidates can be synthesized into practical FPGA-resident checkers.

Chapter 5

hls4ml Synthesis and Hardware Results

5.1 Synthesis Goal

The CNN models from Chapter 4 are useful only if they can be mapped to practical FPGA hardware. The goal of this chapter is to evaluate whether the trained classifiers can become deployment-path checkers with acceptable latency and resource overhead.

We use hls4ml to convert trained CNN models into hardware designs [8, 9]. The synthesis flow takes a trained Keras or QKeras model [30], generates an HLS implementation, validates the generated design with C simulation, and produces latency and resource estimates after synthesis. The final W8A8 P50 manual-v1 candidates are trained with QKeras quantization-aware training and 50% pruning before hls4ml conversion, so the software model already uses the quantized weight and activation behavior evaluated by the hardware flow. The final candidates are then evaluated in terms of added latency and added FPGA resources relative to the Coyote baseline.

The main hardware metrics are LUT usage, BRAM usage, DSP usage, and added inspection latency. The main detection metrics are accuracy, F1, true positive rate, false positive rate, and ROC AUC. The final design point should balance both groups of metrics.

5.2 hls4ml Flow

The hardware flow starts from a trained CNN. The model is exported and passed to hls4ml. hls4ml generates C++ code and an HLS project. We then run C simulation to check that the generated model produces outputs consistent with the software model. After this parity check, the design is synthesized and evaluated.

The flow is:

trained CNN → hls4ml conversion → C simulation
→ HLS synthesis → latency and resource report

This flow is repeated for selected CNN candidates. We do not synthesize every software model. Larger software models can improve F1, but they can also produce hardware designs that are too slow or too resource-heavy. The synthesis stage therefore acts as a filter between high-quality software models and deployable FPGA checkers.

5.3 Hardware Tuning Parameters

The main hls4ml tuning parameters are quantization, pruning, reuse factor, synthesis strategy, and manual per-layer configuration.

Quantization controls the precision of weights and activations. Lower precision can reduce hardware cost, but it can also reduce model quality or create numerical differences between the software model and the generated HLS model.

Pruning controls weight sparsity. In the final W8A8 P50 candidates, P50 denotes a final sparsity target of 0.5 for pruned layers.

The reuse factor controls how much hardware is shared across operations. A low reuse factor increases parallelism and reduces latency. It also increases resource usage. A high reuse factor uses fewer resources but increases latency.

The synthesis strategy controls whether hls4ml optimizes more aggressively for latency or resource usage. We also use manual per-layer tuning because a single global setting is often too coarse. Some layers dominate latency, while others dominate BRAM or DSP usage.

Table 5.1 summarizes the main knobs.

5.4 Latency and Resource Tradeoff

The HLS exploration shows a clear tradeoff between detection quality, latency, and resource usage. Larger input resolutions preserve more bitstream detail and improve software F1. They also increase the cost of the hardware checker. Lower reuse factors reduce latency but require more parallel hardware. Higher reuse factors reduce hardware use but increase latency.

Figure 5.1 shows the W8A8 P50 resource-strategy sweep. Circles and squares show reuse-factor sweeps for the 256×7 and 512×7 models. The starred points show the manually tuned v1 candidates used for deployment evaluation. These manual candidates keep the same reported F1 as the corresponding resource-strategy sweep while reducing latency and LUT utilization.

Table 5.1: hls4ml tuning parameters used during synthesis.

Parameter	**Effect**	**Main tradeoff**
Quantization	Sets weight and activation precision	Quality vs. area
Pruning	Sets target weight sparsity	Quality vs. exploitable sparsity
Reuse factor	Controls hardware sharing across operations	Latency vs. area
Strategy	Selects resource-oriented or latency-oriented mapping	Area vs. speed
Per-layer tuning	Adjusts important layers individually	Practical fit

Table 5.2: Final synthesized candidates. Resource percentages are routed hls4ml wrapper/user-logic attribution on the target U55C.

Model	**Acc.**	**F1**	**TPR**	**TPR Ex.**	**FPR**	**AUC**	**Latency**	**LUT**	**BRAM**	**DSP**
256×7 W8A8 P50 manual-v1	87.7%	88.5%	92.0%	98.7%	16.9%	95.1%	3.35 ms	+4.47%	+17.86%	+14.72%
512×7 W8A8 P50 manual-v1	90.4%	90.5%	89.3%	100.0%	8.45%	96.9%	6.53 ms	+4.90%	+61.78%	+14.45%

Figure 5.2 and Figure 5.3 show the broader design-space frontier. Unless the design is hand-optimized per CNN layer, better F1 generally requires a larger latency budget. The same pattern appears for LUT utilization. Higher-quality models are available, but they move rightward on the latency and LUT axes. The hand-optimized points are important because they shift selected candidates toward the useful frontier instead of accepting the default global resource-strategy mapping.

5.5 Final Synthesized Candidates

The final comparison focuses on two synthesized models. The final 256×7 candidate is the seven-layer, W8A8, 50%-pruned model with the manually tuned v1 hls4ml configuration. It is the preferred deployment candidate. The final 512×7 candidate uses the same W8A8 P50 manual-v1 setup at higher input resolution. It is the accuracy-oriented alternative.

Table 5.2 shows the final comparison. Classification metrics are evaluated on the original small-RO test set unless stated otherwise. The extended TPR is measured on the extended RO-footprint dataset.

The 512×7 W8A8 P50 manual-v1 model has higher accuracy, higher F1,

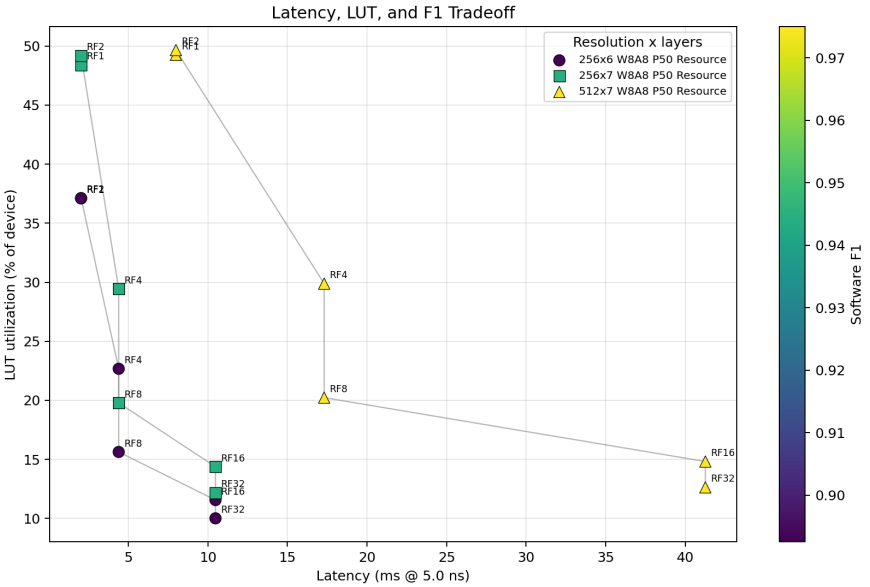


Figure 5.1: Latency, LUT utilization, and F1 tradeoff for W8A8 P50 resource-strategy candidates. Marker shapes distinguish the 256×7 and 512×7 reuse-factor sweeps from the manually tuned v1 deployment candidates.

lower false positive rate, higher ROC AUC, and perfect TPR on the extended RO-footprint dataset. This makes it the stronger detection candidate.

The 256×7 W8A8 P50 manual-v1 model is the better deployment candidate. It has lower added latency and much lower BRAM overhead. Its F1 is only two percentage points lower than the 512×7 W8A8 P50 manual-v1 model, and its original-set TPR is higher. The BRAM difference is the main reason for preferring 256×7. The routed hierarchy attributes 61.78% of device BRAM to the 512×7 checker wrapper, while the 256×7 wrapper accounts for 17.86%.

5.6 Latency Context

The added inspection latency should be interpreted relative to the latency of partial reconfiguration and serverless invocation. Table 5.3 compares the 256×7 and 512×7 inspection latencies with F3 and generic serverless reference points [14, 31].

The added latency is small relative to full serverless invocation latencies and larger PR operations. It is more significant for very small PR regions. This suggests that FPGA-resident inspection is most attractive when the checker is used in realistic deployment paths where PR and runtime overheads already dominate a few milliseconds of inference time.

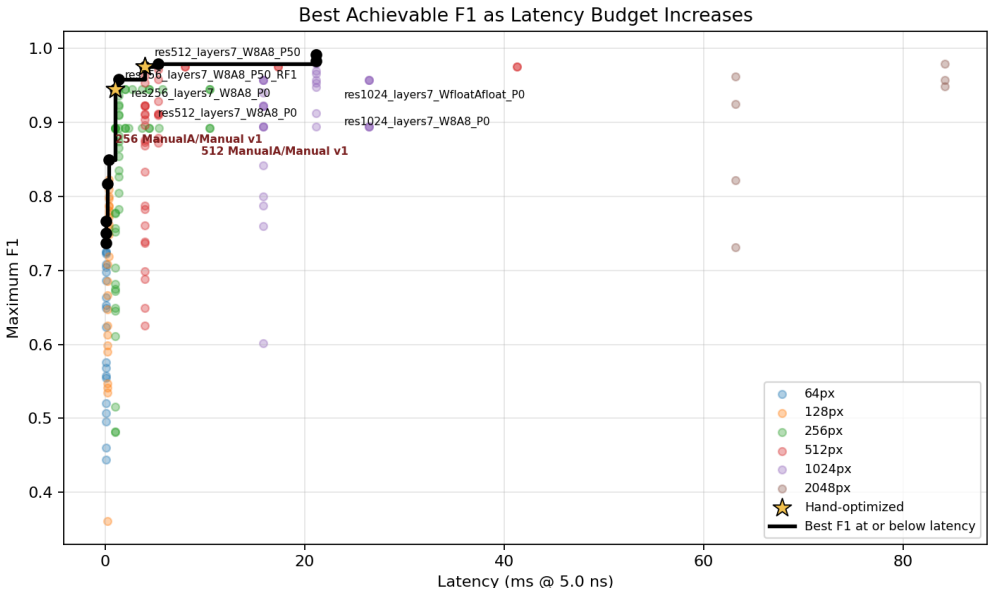


Figure 5.2: Best achievable F1 as the latency budget increases. The black frontier shows the maximum F1 achieved by any synthesized candidate within each latency budget. Stars indicate the manually optimized deployment candidates.

The 256×7 W8A8 P50 manual-v1 model adds 3.35 ms. The 512×7 W8A8 P50 manual-v1 model adds 6.53 ms. Both are plausible for deployment-path inspection, but the smaller model gives a safer latency budget.

5.7 Resource Context

The baseline Coyote design uses 10.26% of LUTs and 9.77% of BRAM on the target device. It uses no URAM or DSPs in the reported baseline. The checker adds resources on top of this baseline.

Table 5.4 summarizes the baseline and the added checker overhead.

The LUT overhead is similar for both synthesized candidates. DSP overhead is also similar. BRAM is the limiting resource. This makes the 256×7 candidate more attractive for integration because it leaves more memory resources available for the shell, services, and user logic.

5.8 Quantization and Pruning

Quantization and pruning are evaluated as hardware-aware compression techniques [30, 32]. Quantization reduces arithmetic precision. Pruning removes

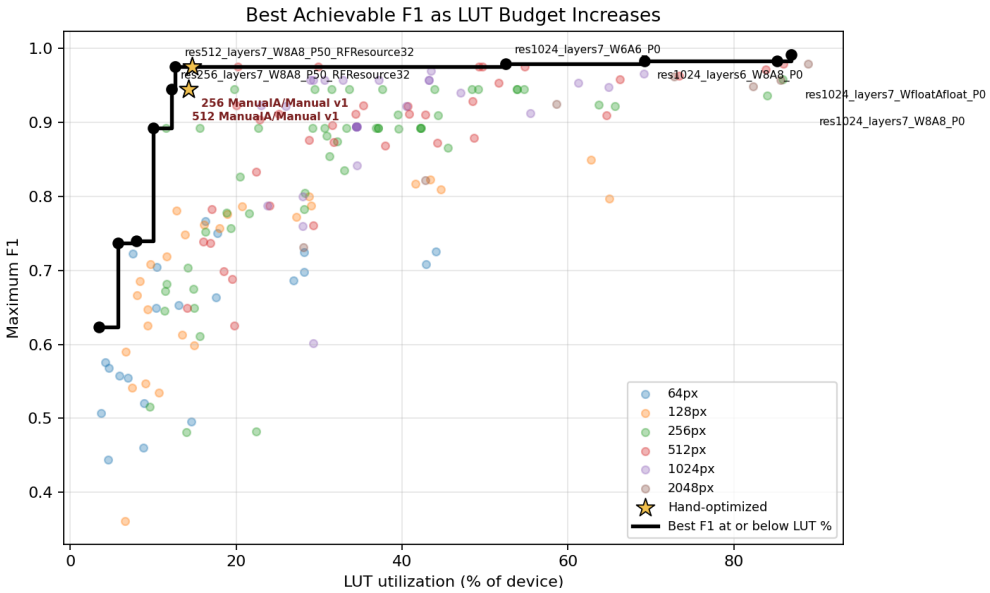


Figure 5.3: Best achievable F1 as the LUT budget increases. The black frontier shows the maximum F1 achieved by any synthesized candidate within each LUT budget. Stars indicate the manually optimized deployment candidates.

weights and increases sparsity. Both can reduce the cost of a neural network, but the benefit depends on the synthesis flow and on whether the generated hardware can exploit the reduced precision or sparsity. For the selected W8A8 P50 models, quantization is not applied only after training. We use QKeras quantization-aware training so the model learns under the target 8-bit weight and activation quantizers.

For quantized models, we check both software quality and hls4ml parity. This is important because a low-bit model can have good software accuracy but still produce mismatches after conversion. For pruned models, we track actual sparsity and classification quality. Sparse models are useful only if sparsity translates into lower synthesis cost or better feasibility.

In the final design selection, quantization and pruning are treated as supporting tools. The main deployment decision is driven by the synthesized 256×7 and 512×7 W8A8 P50 manual-v1 candidates, where measured latency and resource overhead are available.

5.9 Takeaway

The hardware results show that FPGA-resident CNN inspection is feasible, but the design point matters. The 512×7 W8A8 P50 manual-v1 model gives stronger

Table 5.3: Added inspection latency relative to deployment and serverless latency references.

Context	**Baseline**	**256×7**	**512×7**
F3 warm path with reconfiguration	141 ms	+2.38%	+4.63%
Coyote PR on VU9P with 6% PR region	6.04 ms	+55.46%	+108.11%
Coyote PR on VU9P with 33% PR region	36.94 ms	+9.07%	+17.68%
Coyote PR on VU9P with 66% PR region	73.27 ms	+4.57%	+8.91%
Generic FaaS cold invocation median	448 ms	+0.75%	+1.46%

Table 5.4: Baseline Coyote resource usage and routed checker wrapper/user-logic attribution.

Design	**LUT**	**BRAM**	**URAM**	**DSP**
Baseline Coyote	10.26%	9.77%	0%	0%
Added 256×7 checker	+4.47%	+17.86%	+1.15%	+14.72%
Added 512×7 checker	+4.90%	+61.78%	+1.77%	+14.45%

detection metrics, especially lower FPR, higher ROC AUC, and perfect TPR on the extended RO-footprint dataset. It also has much higher BRAM overhead and roughly double the latency of the 256×7 W8A8 P50 manual-v1 model.

The 256×7 W8A8 P50 manual-v1 model is the preferred candidate for further Coyote integration. It provides useful detection quality, lower added latency, and much lower BRAM overhead. It is the better balance between security coverage and deployment cost.

Chapter 6

Proposed Coyote Integration

6.1 Integration Point

The final goal is to place the bitstream checker directly on the Coyote deployment path. In this path, a user bitstream reaches the platform before partial reconfiguration. The checker should inspect the bitstream before it is loaded into the reconfigurable region.

Coyote provides a suitable target because it already separates platform services from user logic [15, 16]. The static shell manages host communication, memory services, networking, command handling, and runtime control. User accelerators are deployed into reconfigurable regions. This structure gives a clear location for inspection before the partial reconfiguration step.

Figure 6.1 shows the proposed placement. The checker receives the user partial bitstream, applies preprocessing, runs the synthesized CNN, and returns a decision to the deployment logic.

6.2 Checker Flow

The checker follows the same pipeline used during model evaluation. The incoming partial bitstream is treated as a byte stream. The preprocessing logic resizes the stream into the fixed tensor shape expected by the CNN. The synthesized hls4ml model then produces a maliciousness score or binary prediction.

The deployment logic uses this output to control partial reconfiguration. If the score is below the rejection threshold, the bitstream is allowed to continue to the PR controller. If the score is above the threshold, the platform blocks the reconfiguration and logs the event.

The intended flow is:

user bitstream → preprocessing → CNN checker
→ allow or reject → partial reconfiguration

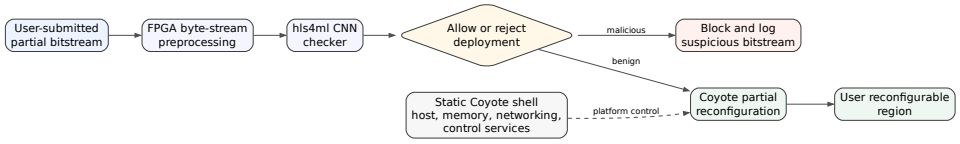


Figure 6.1: Proposed Coyote integration path. The partial-bitstream checker is placed before partial reconfiguration and decides whether the bitstream should be loaded or rejected.

This design keeps inspection inside the FPGA deployment path. It avoids a separate host-side inspection stage and gives the platform a self-contained gate before loading user logic. This follows the broader goal of making FPGA-resident ML components deployable inside systems paths rather than treating them only as offline models [33].

6.3 Prototype Scope

This work evaluates the main components needed for such an integration. The preprocessing method has already been implemented as HLS C++ raw-byte downsampling logic, lowered to Verilog by the HLS flow, and used for the deployment-oriented latency results. The CNN candidates are synthesized with hls4ml and evaluated for latency and resource cost.

The current integration is still a proposal and prototype direction. A complete Coyote deployment would require control-plane integration, threshold policy, logging, error handling, and validation with the real PR controller. It would also need a clear policy for low-confidence predictions.

Chapter 7

Discussion, Limitations, and Future Work

7.1 Discussion

The results show that FPGA-resident ML inspection is a feasible direction for serverless FPGA deployment. The checker can be placed before partial reconfiguration, trained CNNs can classify Coyote-style partial bitstreams with useful accuracy, and hls4ml can synthesize selected models into concrete hardware candidates.

The main design lesson is that the best software model is not always the best deployment model. The 512×7 candidate improves several detection metrics, yet it also has prohibitively higher BRAM overhead and higher latency. The 256×7 candidate gives a better deployment balance. It has lower added latency, much lower BRAM use, and still provides useful detection quality.

The checker should be understood as a defense-in-depth component, affecting and improving the economics of serverless FPGAs. It can reduce the chance that an RO-based malicious bitstream reaches the reconfigurable region. It does not prove that a bitstream is safe. A practical platform should combine this checker with static validation, bitstream authentication, runtime monitoring, and platform policy [22, 23, 34].

The RO-footprint results also show where the detector is strongest. Larger RO payloads are detected more reliably. These payloads are the most relevant for power-stress attacks. Smaller payloads remain harder and require stronger datasets, better models, or additional checks.

7.2 Limitations

This work is a feasibility study. It makes progress toward FPGA-resident inspection, but it has several limitations.

Table 7.1: Current limitations of the prototype.

Limitation	Impact
Binary classification	The current task distinguishes benign Coyote bitstreams from RO-based malicious bitstreams. It does not cover broader malicious-circuit classes.
RO-focused dataset	The malicious class is generated from ring-oscillator payloads. Other attacks may produce different bitstream patterns.
CNN-only evaluation	Other hardware-friendly classifiers may provide better latency or resource tradeoffs.
Low-footprint attacks	Small malicious payloads are harder to detect. Misses concentrate in the smallest RO-footprint regime.
One platform setting	The evaluation focuses on Coyote-style artifacts and one FPGA platform family. Generalization across devices and shells is not yet shown.
Partial integration	The checker is synthesized and placed in a Coyote path, but it is not yet a complete production component.

7.3 Future Work

The next step is broader validation. The dataset should include more attack classes, more benign applications, and harder malicious examples. Important additions include timing attacks, side-channel circuits, covert channels, non-RO power wasters, and mixed benign-plus-malicious designs.

Low-footprint attacks need special attention. A stronger benchmark should include subtle payloads with varied RO densities and placements. This would test whether the classifier can detect malicious structure without relying on large payload size.

Future work should also evaluate other model families. MLPs, decision trees, tiny CNNs, binary neural networks, and FINN-style quantized models may provide better latency and resource tradeoffs [35, 30, 32]. Netlist-oriented GNN classifiers are another possible direction for broader malicious-circuit coverage [36]. These models should be compared under the same dataset, metrics, and synthesis flow.

Generalization is another open question. The current results should be repeated across different FPGA families, floorplans, shells, and benign workloads.

Versal or V80-class devices would be especially useful targets because they are relevant to newer datacenter FPGA deployments.

The final implementation step would be full Coyote integration. The checker should be connected directly to the PR control path, with threshold policy, logging, error handling, and a clear response for low-confidence predictions. This would turn the current prototype into a complete in-path security gate.

Chapter 8

Conclusion

This project studied whether ML-based malicious-bitstream inspection can move into the FPGA deployment path for serverless FPGA systems. The target setting is a Coyote-style environment where user partial bitstreams are checked before partial reconfiguration. The goal is to add a hardware-backed security gate that can reject suspicious bitstreams before they enter the reconfigurable region.

We built a partial-bitstream inspection pipeline around three components. First, we generated a dataset with benign Coyote application bitstreams and RO-based malicious bitstreams. Second, we transformed each bitstream into a fixed-size byte-image tensor and trained CNN classifiers for binary detection. Third, we synthesized selected CNNs with hls4ml and evaluated their detection quality, latency, and FPGA resource overhead.

The results show that FPGA-resident ML inspection is feasible as a defense-in-depth mechanism. The 512×7 W8A8 P50 manual-v1 model gives stronger accuracy-oriented metrics, including higher F1, lower false positive rate, higher ROC AUC, and perfect detection on the extended RO-footprint dataset. The 256×7 W8A8 P50 manual-v1 model is the preferred deployment candidate because it gives a better hardware tradeoff. It achieves useful detection quality with 3.35 ms added latency and much lower BRAM overhead than the 512×7 alternative.

The main takeaway is that deployment-oriented bitstream inspection must be evaluated across both ML and systems metrics. Software accuracy alone is not enough. A practical checker must also fit beside the FPGA shell and user logic, add limited latency to the PR path, and integrate cleanly into the deployment flow. This prototype shows a practical path toward such a checker, while leaving broader attack coverage, harder datasets, cross-platform validation, and full Coyote integration as future work.

Bibliography

[1] D. R. E. Gnad, F. Oboril, and M. B. Tahoori, "Voltage drop-based fault attacks on fpgas using valid bitstreams," in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*, 2017, pp. 1–7.

[2] T. La, K. D. Pham, J. Powell, and D. Koch, "Denial-of-service on fpga-based cloud infrastructures: Attack and defense," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 3, pp. 441–464, 2021.

[3] G. Provelengios, D. Holcomb, and R. Tessier, "Power wasting circuits for cloud fpga attacks," in *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*, 2020.

[4] J. Rettkowski, S. Mahmood, A. Shallufa, M. Hübner, and D. Göhringer, "Inspection of partial bitstreams for fpgas using artificial neural networks," in *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2019, pp. 83–86.

[5] J. Chaudhuri and K. Chakrabarty, "Detection of malicious fpga bitstreams using cnn-based learning," in *2022 IEEE European Test Symposium (ETS)*, 2022, pp. 1–2.

[6] R. Elnaggar, J. Chaudhuri, R. Karri, and K. Chakrabarty, "Learning malicious circuits in fpga bitstreams," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 3, pp. 726–739, 2023.

[7] A. Palumbo, L. Cassano, B. Luzzi, J. A. Hernández, P. Reviriego, G. Bianchi, and M. Ottavi, "Is your fpga bitstream hardware trojan-free? machine learning can provide an answer," *Journal of Systems Architecture*, vol. 128, p. 102543, 2022.

[8] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, and Z. Wu, "Fast inference of deep neural networks in fpgas for particle physics," *Journal of Instrumentation*, vol. 13, no. 07, p. P07027, 2018.

[9] T. Aarrestad, V. Loncar, N. Ghielmetti, M. Pierini, S. Summers, J. Ngadiuba, C. Petersson, H. Linander, Y. Iiyama, G. Di Guglielmo, J. Duarte, P. Harris, D. Rankin, S. Jindariani, K. Pedro, N. Tran, M. Liu, E. Kreinar, Z. Wu, and D. Hoang, "Fast convolutional neural networks on fpgas with

hls4ml," *Machine Learning: Science and Technology*, vol. 2, no. 4, p. 045015, 2021.

[10] A. M. Caulfield, E. S. Chung, A. Putnam, H. Angepat, J. Fowers, M. Haselman, S. Heil, M. Humphrey, P. Kaur, J.-Y. Kim, D. Lo, T. Massengill, K. Ovtcharov, M. Papamichael, L. Woods, S. Lanka, D. Chiou, and D. Burger, "A cloud-scale acceleration architecture," in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*, 2016, pp. 1–13.

[11] E. S. Chung, J. Fowers, K. Ovtcharov, M. K. Papamichael, A. M. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, K. Husseini, T. Kagi, R. Kovvuri, S. Lanka, D. Mukhortov, P. Patel, B. Perez, A. Rapsang, S. Reinhardt, A. Sapek, R. Seefeldt, H. Tian, G. Tiong, G. Uyeda, G. Weisz, L. Woods, P. Yi, D. Zhou, and D. Burger, "Serving dnns in real time at datacenter scale with project brainwave," *IEEE Micro*, vol. 38, no. 2, pp. 8–20, 2018.

[12] C. Bobda, J. M. Mbongue, P. Chow, M. Ewais, N. Tarafdar, J. C. Vega, K. Eguro, D. Koch, S. Handagala, M. Leeser, M. Herbordt, H. Shahzad, P. Hofste, B. Ringlein, J. Szefer, A. Sanaullah, and R. Tessier, "The future of fpga acceleration in datacenters and the cloud," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 3, pp. 1–42, 2022.

[13] X. Wang, Y. Niu, F. Liu, and Z. Xu, "When fpga meets cloud: A first look at performance," *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 1344–1357, 2022.

[14] C. Mainas, M. Lambeck, B. Scheufler, L. Bindschaedler, A. Koshiba, and P. Bhatotia, "F3: An fpga-accelerated faas framework," in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, 2025.

[15] D. Korolija, T. Roscoe, and G. Alonso, "Do os abstractions make sense on fpgas?" in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 991–1010.

[16] B. Ramhorst, D. Korolija, M. J. Heer, J. Dann, L. Liu, and G. Alonso, "Coyote v2: Raising the level of abstraction for data center fpgas," in *Proceedings of the ACM Symposium on Operating Systems Principles*, 2025.

[17] T. M. La, K. Mätas, N. Grunchevski, K. D. Pham, and D. Koch, "Fpgadefender: Malicious self-oscillator scanning for xilinx ultrascale+ fpgas," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 13, no. 3, pp. 15:1–15:31, 2020.

[18] A. Boutros, M. Hall, N. Papernot, and V. Betz, "Neighbors from hell: Voltage attacks against deep learning accelerators on multi-tenant fpgas," in *2020 International Conference on Field-Programmable Technology (ICFPT)*, 2020, pp. 103–111.

[19] D. G. Mahmoud, O. Glamočanin, F. Regazzoni, and M. Stojilović, "Practical implementations of remote power side-channel and fault-injection attacks on multitenant fpgas," in *Security of FPGA-Accelerated Cloud Computing Environments*. Springer, 2023, pp. 101–135.

[20] Amazon Web Services, "Amazon ec2 f2 instances," https://aws.amazon.com/ec2/instance-types/f2/, 2026, accessed 2026-07-14.

[21] J. Chaudhuri and K. Chakrabarty, "Criticality analysis of ring oscillators in fpga bitstreams," in *2023 IEEE European Test Symposium (ETS)*, 2023, pp. 1–4.

[22] C. Jin, V. Gohil, R. Karri, and J. Rajendran, "Security of cloud fpgas: A survey," 2020.

[23] J. Zhang and G. Qu, "Recent attacks and defenses on fpga-based systems," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 3, 2019.

[24] I. Giechaskiel, K. B. Rasmussen, and K. Eguro, "Measuring long wire leakage with ring oscillators in cloud fpgas," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, 2019.

[25] J. Yoon, Y. Seo, J. Jang, M. Cho, J. Kim, H.-M. Kim, and T. Kwon, "A bitstream reverse engineering tool for fpga hardware trojan detection," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2318–2320.

[26] R. Stahle-Smith and R. Karakchi, "Real-time ml-based defense against malicious payload in reconfigurable embedded systems," 2025. [Online]. Available: https://arxiv.org/abs/2509.02387

[27] M. Chen and P. Liu, "A deep learning-based fpga function block detection method with bitstream to image transformation," 2020.

[28] J. Chaudhuri and K. Chakrabarty, "Detection and classification of malicious bitstreams for fpgas," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, 2023.

[29] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[30] C. N. Coelho Jr., A. Kuusela, H. Zhuang, T. Aarrestad, V. Loncar, J. Ngadiuba, M. Pierini, S. Summers, G. Di Guglielmo, J. Duarte *et al.*, "Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors," 2020.

[31] D. Ustiugov, T. Amariucai, and B. Grot, "Analyzing tail latency in serverless clouds with stellar," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 51–62.

[32] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.

[33] M. J. Heer, B. Ramhorst, and G. Alonso, "Easing the path to deployment in ml4sys through fpgas," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*, 2025, pp. 56–62.

[34] M. K. Ahmed, J. Mandebi, S. K. Saha, and C. Bobda, "Multi-tenant cloud fpga: A survey on security," 2022.

[35] M. Blott, T. B. Preußer, N. J. Fraser, G. Gambardella, K. O'Brien, Y. Umuroglu, M. Leeser, and K. Vissers, "Finn-r: An end-to-end deep-learning framework for fast exploration of quantized neural networks," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 11, no. 3, 2018.

[36] L. Alrahis, H. Nassar, J. Krautter, D. Gnad, L. Bauer, J. Henkel, and M. Tahoori, "Malignnoma: Gnn-based malicious circuit classifier for secure cloud fpgas," 2024.